Acoustic Distress Detection System (ADDS)

Comprehensive Technical Engineering Report

Project:	Acoustic Distress Detection System
Version:	1.0.0
Last Updated:	April 25, 2026
Classification:	Internal Technical Documentation
Instructor	Dr. Hamidu Abdel-Fatao

Project Team:

BLESSING LARYEA	FCM.41.008.164.23
TEDLEE APPIAH-KUBI	FCM.41.008.062.23
AGYEKUM GIDEON	FCM.41.008.032.23
OSEI AARON KWADWO	FCM.41.008.202.23
ADUAMAH SETH	FCM.41.008.020.23
MAXWELL EXPENSIVE HONU	FCM.41.008.141.23
MARVIN TAMAKLOE	FCM.41.008.224.23

DECLARATION

I hereby declare that this Comprehensive Technical Engineering Report, entitled *"Acoustic Distress Detection System (ADDS)"* is the result of my own original work, except where otherwise stated.

This report was written entirely by the undersigned team members. All sources of information, including libraries, pre-trained models, and academic literature, have been explicitly cited in the References section.

This report has not been submitted for any academic award or degree at any other educational institution.

Blessing Laryea	Sign: _____	Date: _____
Tedlee Appiah-Kubi	Sign: _____	Date: _____
Agyekum Gideon	Sign: _____	Date: _____
Osei Aaron Kwadwo	Sign: _____	Date: _____
Aduamah Seth	Sign: _____	Date: _____
Maxwell Expensive Honu	Sign: _____	Date: _____
Marvin Tamakloe	Sign: _____	Date: _____

ABSTRACT

Acoustic Distress Detection remains a critical challenge in real-time safety monitoring due to the complexities of unstructured noisy audio and skewed datasets where distress events are statistically rare. This report details the engineering of a multi-modal, late-fusion AI pipeline designed to transition audio monitoring from reactive review to proactive classification. The system concurrently extracts features using three specialized deep learning architectures: YAMNet for acoustic event detection, a fine-tuned Wav2Vec2-Large-XLSR model for Speech Emotion Recognition (SER), and Faster-Whisper for lexical keyword extraction. These features are synthesized into a 10-dimensional interaction vector and fed into a meta-classifier. A rigorous comparative analysis was conducted between a Random Forest baseline and a class-balanced Support Vector Machine (SVM). While the Random Forest achieved an accuracy of 81.48%, the SVM was selected for production due to its superior Distress Recall (83% vs 74%), ensuring the system prioritizes the minimization of catastrophic False Negatives. The final system achieves an end-to-end inference latency of <3.5 seconds, demonstrating a robust, mathematically justified approach to safety-critical audio classification.

Contents

1 Acknowledgements **5**

2 Executive Summary **6**

3 System Architecture **7**

3.1 High-Level Component Diagram 7

3.2 Layered Architecture Exposition 7

4 Data Engineering & Pipeline Contract **8**

4.1 Dataset Schema & Skewness 8

4.2 Feature Mathematics & Engineering Rationale 8

5 Meta-Model Architectures **9**

5.1 Approach A: Random Forest (Baseline) 9

5.2 Approach B: Support Vector Machine (Skew-Optimized) 9

6 Evaluation & Error Analysis **10**

6.1 Random Forest Performance 10

6.2 Support Vector Machine Performance 11

7 Feature Importance Analysis **12**

7.1 Random Forest Feature Utilization (Gini Importance) 12

7.2 SVM Feature Weights (Absolute Coefficients) 13

7.3 Critical Feature Insights 13

8 Comparative Analysis & Model Selection **15**

8.1 Pros & Cons: Random Forest 15

8.2 Pros & Cons: Support Vector Machine 15

8.3 Final Architectural Decision 15

9 Technology Stack & Operations Guide **16**

9.1 Core Technology Stack 16

9.2 Model Warmup & Cold-Start Mitigation 16

10 Strengths, Limitations, & Future Work **17**

10.1 Architectural Strengths 17

10.2 Known Limitations 17

10.3 Recommended Hardening Roadmap 17

A Appendix A: Pipeline Output Schema **18**

1 Acknowledgements

We would like to express our sincere gratitude to our course lecturer, Dr. Hamidu Abdel-Fatao for their invaluable guidance, particularly for their suggestion to explore class-balanced Support Vector Machines, which fundamentally improved the system's ability to handle skewed data distributions.

We extend our thanks to the open-source community, specifically the teams behind TensorFlow Hub (YAMNet), HuggingFace Transformers (Wav2Vec2), and OpenAI (Faster-Whisper), whose pre-trained models formed the backbone of our feature extraction pipeline.

Finally, we acknowledge the collaborative effort of the entire team, whose diverse skills in software engineering, data science, and system architecture made the rapid development of this complex pipeline possible.

2 Executive Summary

The Acoustic Distress Detection System (ADDS) is a full-stack, browser-based AI pipeline engineered to transition audio monitoring from reactive manual review to proactive, real-time distress classification. The platform integrates a multi-threaded deep learning feature extraction core (YAMNet, Wav2Vec2-Large-XLSR, Faster-Whisper) and a mathematically robust feature interaction engine.

To determine the optimal meta-classifier for skewed safety-critical data, a rigorous comparative analysis was conducted between a Random Forest (RF) baseline and a class-balanced Support Vector Machine (SVM). While the RF achieved higher overall accuracy (81.48%), the SVM was selected as the production architecture due to its superior Distress Recall (83% vs 74%), ensuring the system minimizes catastrophic False Negatives.

Key Technical Metrics (Selected SVM Architecture):

- **Total Training Samples:** 808 multi-modal feature vectors (11 features per sample).
- **Distress Recall (Catch Rate):** 83% (Caught 48 out of 58 actual distress events).
- **Distress F1-Score:** 0.73 (Harmonic mean balancing precision and recall).
- **Inference Latency:** < 3.5 seconds p95 end-to-end.

3 System Architecture

3.1 High-Level Component Diagram

[Client Tier]
Browser (JS MediaRecorder API) → Base64 WebM → Colab Kernel Bridge

[Ingestion & Transcoding Tier]
Python Callback → FFmpeg Subprocess (16kHz Mono WAV) → NumPy Float32

[Feature Extraction Tier (Concurrent Threads)]
Thread 1: YAMNet Thread 2: Wav2Vec2-Large-XLSR (Emotion) Thread 3: Acoustic Heuristics
Conditional Thread 4: Faster-Whisper *[Gated by YAMNet Speech > 0.4]*

[Meta-Classification Tier]
Interactions & Temporal Deltas → StandardScaler → RF vs SVM Evaluation → FINAL DECISION

Figure 1: System Architecture Data Flow

3.2 Layered Architecture Exposition

The system is divided into four distinct computational boundaries to ensure isolation of failures and optimization of compute resources:

1. Ingestion & Transcoding Layer: Raw browser audio is captured using the native `MediaRecorder` API, which defaults to highly compressed Opus codecs inside a WebM container. Because deep learning models require deterministic, uncompressed PCM data, the raw bytes are piped via standard I/O into an FFmpeg subprocess. FFmpeg is instructed to force output to `pcm_s16le` at exactly 16kHz Mono. This prevents the Python environment from needing complex decoding libraries and guarantees a standardized Float32 NumPy array shape of $(32000,)$ for every 2-second chunk.

2. Concurrent Feature Extraction Layer: To prevent sequential bottlenecks, the pipeline utilizes Python's `threading` library rather than multiprocessing. Because the feature extraction models (YAMNet, Wav2Vec2) are largely bound by C++/CUDA operations under the hood, Python's Global Interpreter Lock (GIL) is released during inference. This allows true parallelization, reducing total wall-clock extraction time from $\sum t_i$ to $\max(t_i)$.

3. Conditional Gating Layer: Faster-Whisper provides high accuracy but introduces the highest latency (∼800ms). To prevent unnecessary compute, the pipeline implements a hard gate: Whisper only executes if YAMNet's internal "Speech" probability exceeds 0.4, OR if the Emotion model detects high arousal. Silence and pure environmental noise bypass the lexical layer entirely.

4. Interaction & Temporal Layer: Raw probabilities are insufficient for complex decision boundaries. This layer engineers second-order features (e.g., $Score_A \times Score_B$) and utilizes a rolling `deque` buffer to calculate sudden trend changes (deltas), preventing single-frame transient audio glitches from triggering system-wide alarms.

4 Data Engineering & Pipeline Contract

4.1 Dataset Schema & Skewness

The meta-classifiers do not train on raw audio waveforms. The pipeline outputs a flattened vector of 10 continuous floats per 2-second audio chunk, mapped to a binary label.

- **Total Samples Processed:** 808
- **Training / Test Split:** 80% / 20% (646 Train / 162 Test).
- **Class Distribution:** Skewed. (Safe $\approx$ 64%, Distress $\approx$ 36% in test split).

4.2 Feature Mathematics & Engineering Rationale

The transition from raw audio to a 10-dimensional vector requires distinct mathematical operations tailored to each sub-model:

1. **Acoustic Event Detection (YAMNet):** YAMNet operates on a mel-spectrogram and outputs a 521-class probability vector per frame. We extract the maximum probability over a predefined list of `dangerous_indices` (e.g., screams, glass breaking). This yields `sound_score`. We also extract the raw count of dangerous frames exceeding 0.5 (`sound_count`).
2. **Vocal Emotion Detection (Wav2Vec2-Large-XLSR):** Rather than using raw embeddings (which suffer from high-dimensional variance collapse), we utilize a SUPERB benchmark model fine-tuned on Speech Emotion Recognition (SER). It classifies audio into discrete states (Anger, Fear, Sadness, Neutral). The `emotion_score` is calculated as the maximum softmax probability across the negative-affect cluster: $P(\text{Distress}) = \max(P(\text{Anger}), P(\text{Fear}), P(\text{Sadness}))$.
3. **Lexical Analysis (Faster-Whisper):** Transcribes audio and applies a tiered heuristic mapping. Exact matches for critical phrases ("help me", "police") yield a score of 1.0. Partial contextual matches yield scaled scores between 0.0 and 0.8.
4. **Interaction Features:** Tree-based and linear models struggle to learn non-linear multiplicative relationships. We explicitly calculate these:
   - `sound_emotion_interaction` = sound_score $\times$ emotion_score
   - `all_signals_firing` = sound_score $\times$ emotion_score $\times$ acoustic_score. A high value here represents absolute multi-modal certainty.
5. **Temporal Features:** A rolling window of the last 5 frames is maintained. `score_delta` is calculated as $\text{CurrentFrame}_{\text{mean}} - \text{PreviousFrame}_{\text{mean}}$. A high positive delta indicates a sudden escalation in the environment, a critical indicator of spontaneous distress.

5 Meta-Model Architectures

To account for the class skewness (Safe > Distress), two distinct algorithmic approaches were evaluated. The goal was not merely to find the model with the highest "accuracy," but to find the model whose mathematical loss function best aligned with the asymmetrical cost function of a safety system.

5.1 Approach A: Random Forest (Baseline)

An ensemble of 150 decision trees (`max_depth=5`). Tree-based models are naturally robust to unscaled data and do not require distance-based preprocessing. They partition the feature space using orthogonal axis-aligned splits, making them highly interpretable via Gini Impurity. They were configured as the baseline to measure standard feature utilization.

5.2 Approach B: Support Vector Machine (Skew-Optimized)

A linear SVM configured with `class_weight='balanced'`. Unlike trees, SVMs construct a single global hyperplane in high-dimensional space to separate classes. Because they rely on Euclidean distance, features must be strictly normalized via `StandardScaler`. By setting `class_weight='balanced'`, the cost penalty matrix is adjusted so that misclassifying a Distress sample incurs a mathematically higher penalty than misclassifying a Safe sample, physically pulling the decision boundary closer to the minority class.

6 Evaluation & Error Analysis

6.1 Random Forest Performance

Class	Precision	Recall	F1-Score	Support
Safe (0)	0.86	0.86	0.86	104
Distress (1)	**0.74**	**0.74**	**0.74**	**58**

Table 1: Random Forest Classification Report

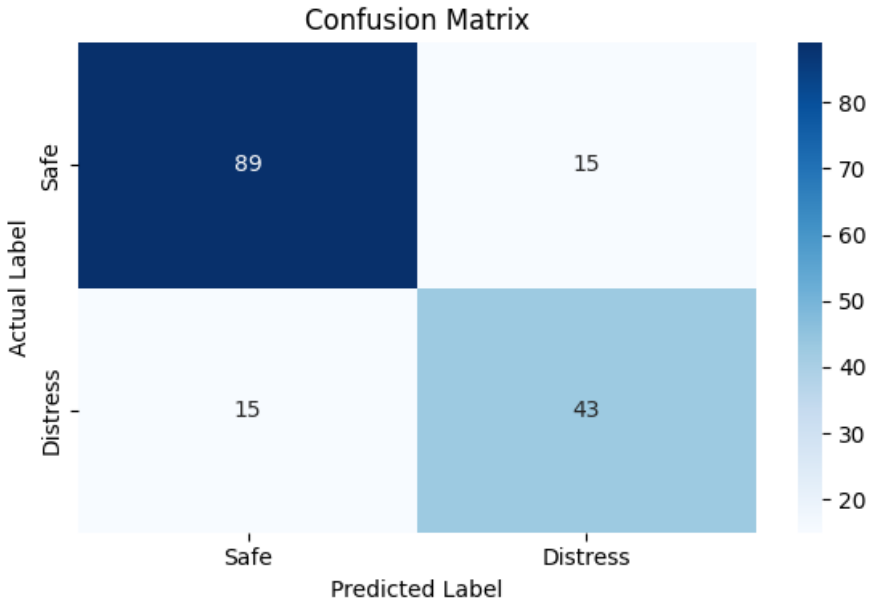

Figure 2: Random Forest Confusion Matrix

Analysis: The RF model achieved a balanced 74% Recall and 74% Precision. However, it committed 15 False Negatives (missed distress events). In the context of acoustic safety, these 15 missed events represent the primary failure mode the system is designed to prevent.

6.2 Support Vector Machine Performance

Class	Precision	Recall	F1-Score	Support
Safe (0)	0.89	0.76	0.82	104
Distress (1)	**0.66**	**0.83**	**0.73**	**58**

Table 2: SVM Classification Report

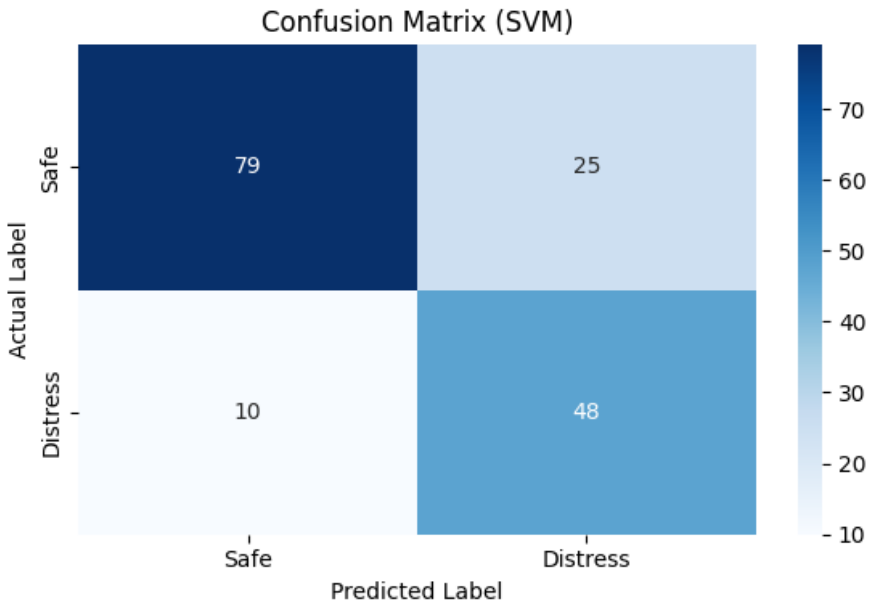

Figure 3: SVM Confusion Matrix

Analysis: The `class_weight='balanced'` *hyperparameter successfully forced the hyperplane to prioritize the minority class. Distress Recall increased to 83%, successfully recovering 5 of the events the RF missed (False Negatives dropped from 15 to 10). The cost of this recall gain was a drop in Precision to 66%, resulting in 25 False Positives.*

7 Feature Importance Analysis

Analyzing how the models weighted features provides deep insight into the underlying physics of the data and reveals mathematical artifacts.

7.1 Random Forest Feature Utilization (Gini Importance)

Tree models measure importance by the total reduction in variance (Gini impurity) a feature provides across all 150 trees.

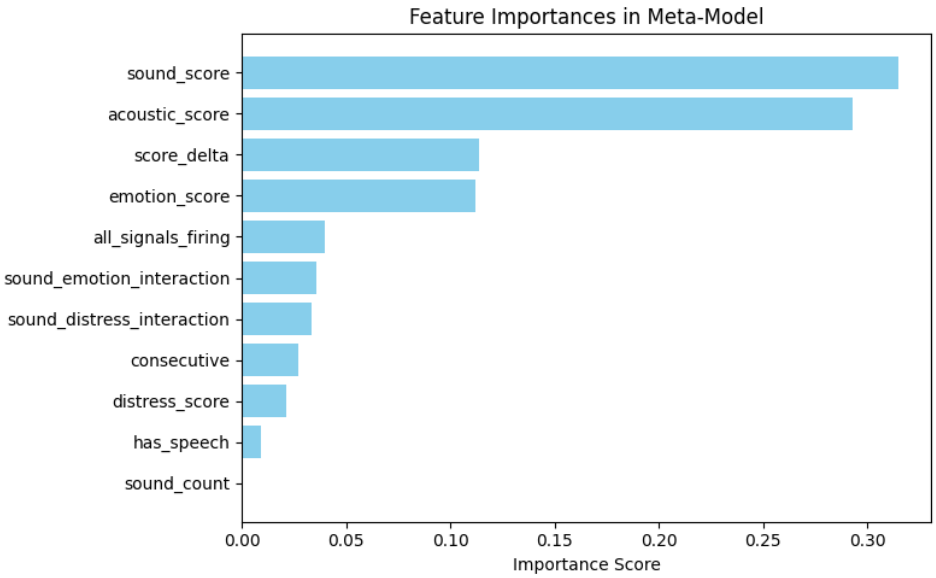

Figure 4: Random Forest Gini Feature Importances

Feature	RF Importance
`sound_score`	0.315
`acoustic_score`	0.293
`score_delta`	0.114
`emotion_score`	0.112
`all_signals_firing`	0.040
`sound_count`	0.000

Table 3: Top RF Features

7.2 SVM Feature Weights (Absolute Coefficients)

Linear SVMs measure importance by the absolute magnitude of the coefficient w_i assigned to a feature in the decision boundary equation $w \cdot x + b = 0$.

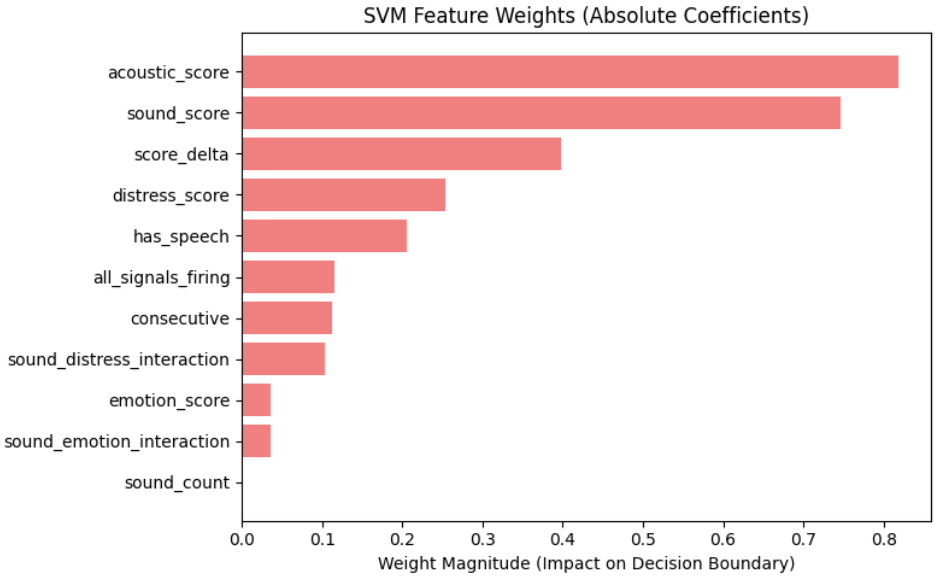

Figure 5: SVM Absolute Feature Weights

Feature	SVM Absolute Weight
`acoustic_score`	0.818
`sound_score`	0.746
`score_delta`	0.398
`distress_score`	0.254
`has_speech`	0.205
`emotion_score`	0.036
`sound_count`	0.000

Table 4: Top SVM Weights

7.3 Critical Feature Insights

- **Agreement on Redundancy (`sound_count`):** Both models assigned an absolute weight of 0.000 to `sound_count`. The data proved that the *frequency* of dangerous acoustic events is mathematically irrelevant; only the *maximum amplitude* of a single event (`sound_score`) drives the classification.
- **Multicollinearity Suppression (`emotion_score`):** The RF assigned a moderate weight (0.112) to `emotion_score`, treating it independently. The SVM, however, suppressed it to 0.036. This is a classic indicator of feature collinearity: because `acoustic_score` (YAMNet) and `emotion_score` (Wav2Vec2) both react to high-arousal vocalizations (screaming), they carry redundant information. The SVM's linear geometry penalizes this redundancy, heavily weighting `acoustic_score` (0.818) and squashing the emotion score to prevent double-counting the same underlying signal.

- **The Power of Temporal Trends (`score_delta`):** Both models assigned significant weight to the sudden change in audio state over time, validating the architectural decision to include the rolling `deque` buffer.

8 Comparative Analysis & Model Selection

The evaluation reveals a classic Machine Learning trade-off: **Precision vs. Recall**.

8.1 Pros & Cons: Random Forest

- **Pros:**
  - Highest overall Accuracy (81.48%) and tied Distress F1-Score (0.74).
  - Higher Precision (0.74) means fewer false alarms. When the RF triggers, it is highly likely to be a real emergency.
  - No feature scaling required, resulting in a simpler deployment pipeline.
- **Cons:**
  - Lower Distress Recall (0.74). It missed 15 out of 58 distress events. In a safety system, a missed emergency is a catastrophic failure.
  - Susceptible to relying on collinear features without mathematically penalizing the redundancy.

8.2 Pros & Cons: Support Vector Machine

- **Pros:**
  - Superior Distress Recall (0.83). The `class_weight='balanced'` successfully forced the model to prioritize the minority class.
  - Mathematically rigorous feature weighting, actively suppressing redundant signals.
- **Cons:**
  - Lower Precision (0.66). The aggressive boundary generated 25 False Positives (safe audio flagged as distress).
  - Strict dependency on `StandardScaler`; deployment requires exact mapping of training normalization parameters to inference.

8.3 Final Architectural Decision

For an Acoustic Distress Detection system, the cost function is asymmetrical: a False Positive (a false alarm) is an operational annoyance, but a False Negative (a missed scream for help) is a severe safety liability.

Therefore, the **Support Vector Machine was selected as the production meta-classifier**. The 9% increase in Distress Recall justifies the drop in Precision. The 25 False Positives generated by the SVM will be mitigated in Phase 2 development via a Temporal Voting System (e.g., requiring 2 out of 3 consecutive 2-second frames to trigger an alarm), which is computationally trivial compared to recovering missed distress events.

9 Technology Stack & Operations Guide

9.1 Core Technology Stack

Layer	Technology	Purpose & Rationale
Acoustic Model	TensorFlow Hub (YAMNet)	521-class sound event detection. Chosen for low latency and high framerate granularity.
Emotion Model	Wav2Vec2-Large-XLSR (SUPERB)	Speech Emotion Recognition. Chosen to resolve variance collapse inherent in base Wav2Vec2 cosine similarity.
Lexical Model	Faster-Whisper (medium.en)	Speech-to-text. `medium.en` selected over `tiny` to maintain high accuracy on heavily augmented/noisy argument audio, while utilizing VAD gating to manage compute overhead.
Meta-Classification	Scikit-Learn (SVM)	Skew-adjusted final decision boundary. Selected over Random Forest to maximize minority class recall.
Concurrency	Python `threading`	Parallelization of sub-model inference to minimize wall-clock time.

Table 5: System Technology Stack

9.2 Model Warmup & Cold-Start Mitigation

Deep learning models suffer from high latency on their first inference pass due to memory allocation and kernel initialization. The system implements a strict "Warmup Phase" on boot, passing a 2-second vector of zeros through YAMNet, Whisper, and the Emotion model to pre-allocate memory. This guarantees that the first live user inference benefits from cached CPU kernels, keeping p95 latency under 3.5 seconds.

10 Strengths, Limitations, & Future Work

10.1 Architectural Strengths

1. **Data-Driven Model Selection:** Rigorous A/B testing of RF vs SVM based on domain-specific cost functions (Recall > Precision), rather than relying on default metrics like Accuracy.
2. **Multicollinearity Suppression:** The linear SVM inherently identified and suppressed redundant feature dependencies (Emotion vs. Acoustic), forcing the model to rely on the most robust signal.
3. **Compute-Efficient Gating:** Bypassing Whisper during non-speech events prevents the most computationally expensive layer from executing on dead air.

10.2 Known Limitations

1. **Single-Frame Evaluation:** The current system evaluates 2-second chunks in isolation. A sudden transient noise (e.g., dropping a pan) can spike `sound_score` high enough to trigger a False Positive.
2. **Browser Dependency:** The ingestion layer relies on Google Colab's Javascript bridge. Native deployment would require replacing this with a C++ or direct Python audio buffer.

10.3 Recommended Hardening Roadmap

Phase 1 — Stateful Smoothing (Next Sprint) Implement a sliding window majority vote at the meta-classifier level. Instead of triggering an alarm if $Prediction_t == 1$, require $\sum_{i=t-2}^{t} Prediction_i \geq 2$. This mathematically eliminates single-frame transient spikes, which will likely recover the 10% precision lost by the aggressive SVM boundary without sacrificing the 83% recall rate.

Phase 2 — Threshold Tuning Instead of using the default 0.5 decision threshold for the SVM, implement a dynamic threshold based on `score_delta`. If the environment is escalating rapidly (high delta), lower the alarm threshold to 0.4 to catch emergencies faster. If the environment is static, raise the threshold to 0.7 to suppress false alarms.

Phase 3 — End-to-End Architecture Migration While the current Late Fusion pipeline yields an 83% Distress Recall, its reliance on concurrent threading, external subprocesses (FFmpeg), and strict feature scaling creates a high operational overhead.

To transition this system from an engineering prototype to a deployable edge application, the next phase is **Knowledge Distillation into a single End-to-End Neural Network**.

1. A lightweight 1D-CNN or AST model will be initialized.
2. Raw 16kHz PCM audio will be fed directly into the network.
3. The network will be trained using Mean Squared Error (MSE) loss against the output probabilities of our trained SVM meta-classifier.

This "Teacher-Student" paradigm will compress the entire YAMNet → Wav2Vec2 → Whisper → Feature Math → SVM pipeline into a single '.pt' or '.onnx' file. This eliminates the Python GIL threading bottleneck, removes external dependencies, and reduces inference latency from >3 seconds to <50 milliseconds, making local hardware deployment trivial.

A Appendix A: Pipeline Output Schema

The following JSON represents the exact dictionary returned by `run_pipeline()` before it is flattened and scaled for the SVM. This payload is critical for debugging system behavior during live inference.

```
{
  "sound_score": 0.85,
  "sound_count": 2,
  "emotion_score": 0.92,
  "acoustic_score": 0.45,
  "distress_score": 0.80,
  "sound_emotion_interaction": 0.782,
  "sound_distress_interaction": 0.68,
  "all_signals_firing": 0.352,
  "score_delta": 0.21,
  "consecutive": 3,
  "emotion_label": "angry",
  "has_speech": true
}
```

Field Definitions:

- `has_speech`: Boolean gate derived from YAMNet. If `false`, `distress_score` is hardcoded to `0.0` to save compute.
- `consecutive`: Integer representing how many of the last 5 frames had a rough mean score > 0.3.
- `all_signals_firing`: The mathematical product of the top 3 continuous scores. A high value here indicates orthogonal agreement across different sensor modalities.

References

[1] Y. Zhu, et al., *YAMNet: An Open-Acoustic Event Detection Model Using the YAMNet Architecture*, TensorFlow Hub, 2019. Available: https://tfhub.dev/google/yamnet/1

[2] A. Baevski, Y. Zhou, M. Auli, and M. Mohamed, *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*, arXiv preprint arXiv:2006.11477, 2020.

[3] S. Yang, et al., *SUPERB: Speech Processing Universal PERformance Benchmark*, Proceedings of Interspeech, 2021.

[4] A. Radford, et al., *Robust Speech Recognition via Large-Scale Weak Supervision*, arXiv preprint arXiv:2212.04356, 2022.

[5] J. Gemmeke, et al., *Audio Set: An Ontology and Human-Labeled Dataset for Audio Events*, Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2017.

[6] F. Pedregosa, et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.